



A review on multi-fidelity hyperparameter optimization in machine learning

Jonghyeon Won ^a, Hyun-Suk Lee ^b, Jang-Won Lee ^a

^a School of Electrical and Electronic Engineering, Yonsei University, Seoul, Republic of Korea

^b Department of Artificial Intelligence and Robotics, Sejong University, Seoul, Republic of Korea

ARTICLE INFO

Keywords:

Fidelity
Hyperparameter
Hyperparameter optimization (HPO)
Machine learning
Multi-fidelity HPO

ABSTRACT

Tuning hyperparameters effectively is crucial for improving the performance of machine learning models. However, hyperparameter optimization (HPO) often demands significant computational budget, which is typically limited. Therefore, efficiently using this constrained budget is critical in HPO. *Multi-fidelity HPO* has emerged as a potential solution to this issue. This paper presents a comprehensive review of multi-fidelity HPO in machine learning, discusses recent algorithms for HPO, and proposes directions for future research.

1. Introduction

Machine learning has become increasingly popular owing to its applications across diverse fields, such as computer vision, natural language processing [1], communication networks [2,3], image tracking [4], healthcare [5], and agriculture [6]. A learning model should be trained updating its learning parameters, such as weights and biases, to align with a given data set. However, unlike the learning parameters, parameters such as the number of hidden layers in a neural network, the depth of a decision tree, and learning rate for gradient descent are not updated during training. These *hyperparameters* [7], which define the structure of the model and its training algorithm, must be set before training begins. Hyperparameters directly affect the performance of machine learning models, making it important to determine their optimal values. Typically, hyperparameters are divided into two categories. Model hyperparameters such as the number of hidden layers in a neural network, the depth of a decision tree, or kernel size in a convolutional neural network (CNN) define the architecture of a learning model. Algorithm hyperparameters specify the details of a learning algorithm, such as the learning rate or batch size in mini-batch gradient descent [8].

A machine learning model typically contains multiple hyperparameters, collectively referred to as a hyperparameter configuration. The performance of the machine learning model depends on the value of the chosen hyperparameter configuration, which is why *hyperparameter optimization (HPO)* is crucial to enhance its overall performance [9,10]. Performance evaluations of numerous hyperparameter configurations are essential for HPO. However, since this evaluation involves the training process, it is highly time-consuming. As a result, HPO algorithms require significant computational time and budget [11].

To manage the performance evaluation time in HPO, it is important to understand the concept of the evaluation budget, which includes factors such as the number of epochs for training, size of the training data set, and number of features used for training [12]. For instance, when the performance evaluation for a hyperparameter configuration consumes a large budget, such as through many training epochs, the resulting performance value is typically more accurate due to its convergence. However, this process requires more computational time and is referred to as high-fidelity performance evaluation. Conversely, low-fidelity evaluations use a small budget, resulting in less reliable performance values but requiring less computational time [13–15]. In other words, this creates a trade-off between the performance evaluation accuracy and required computational budget, depending on the fidelity level. As the total computational budget available for HPO is limited, it is imperative to balance the desired accuracy of performance estimates with the need to conserve the budget for multiple hyperparameter configurations.

By adjusting the computational budget for each hyperparameter configuration, the time for the performance evaluation of hyperparameter configurations can be controlled. *Multi-fidelity HPO*, which considers this trade-off among fidelity levels offers a promising solution to resolve the time-consuming nature of HPO [12]. Numerous methods for multi-fidelity HPO have been proposed to efficiently allocate the computational budget across various hyperparameter configurations. To understand multi-fidelity HPO better, we must initially have a thorough understanding of the motivation and principles behind existing works.

Several survey papers [7,8,10,11,16,17] have reviewed HPO methods in machine learning and analyzed the strengths and drawbacks

* Corresponding authors.

E-mail addresses: wonjonghyeon@yonsei.ac.kr (J. Won), hyunsuk@sejong.ac.kr (H.-S. Lee), jangwon@yonsei.ac.kr (J.-W. Lee).

of various approaches. Among these, *Bayesian optimization (BO)*-based methods [18] are particularly popular. Wu et al. [19] analyzed in detail BO algorithms and provided a comprehensive overview of the methodology for applying BO to HPO. Other studies [20,21] have explored multi-objective HPO, which seeks to optimize multiple objective performance functions, while others have focused on a special form, such as neural architecture search (NAS) [22]. Studies focusing on NAS specifically address finding the optimal neural network architecture by determining the model hyperparameters of the network. Studies [23–27] have provided overall surveys on NAS.

Despite such extensive literature on HPO, to the best of our knowledge, none of the existing papers provides an overall review of multi-fidelity HPO. As described above, it is crucial to incorporate multiple fidelity levels in HPO. Therefore, this paper provides a comprehensive review on multi-fidelity HPO methods. Our contributions can be summarized as follows:

- We emphasize the importance of considering multiple fidelity levels in HPO. To this end, we review the general concepts of fidelity and multi-fidelity optimizations, analyze their significance and benefits, and outline basic approaches to offer their thorough understanding.
- We explore current research on multi-fidelity HPO, focusing on the extension of conventional HPO methods to multi-fidelity approaches. Such methods are classified according to their underlying HPO strategies, with explanations of motivations and development processes.
- Despite the progress in multi-fidelity HPO, certain practical aspects remain unexplored. We analyze these unaddressed issues and propose directions for research to further advance the practical use of multi-fidelity HPO algorithms.

The rest of the paper is organized as follows. In Section 2, we briefly provide the preliminary background required for understanding multi-fidelity HPO. Section 3 provides a comprehensive review of existing research on multi-fidelity HPO. Future directions for research are detailed in Section 4. Finally, Section 5 concludes the paper.

2. Preliminaries

To analyze works on multi-fidelity HPO, it is necessary to understand some fundamental concepts of HPO and multi-fidelity HPO.

2.1. Hyperparameter optimization

In this subsection, we outline the general procedure for HPO and its mathematical expression, while also analyzing primary difficulties in developing an algorithm for HPO. We also introduce well-known basic approaches for HPO.

2.1.1. Hyperparameter optimization problems

A machine learning model generally contains numerous hyperparameters. The values of hyperparameters that are controlled via HPO are designated as $\theta_1, \theta_2, \dots, \theta_n$, where n is the total number of hyperparameters. Then, a hyperparameter configuration is defined as a tuple of these hyperparameters, denoted by

$$\Theta = (\theta_1, \theta_2, \dots, \theta_n).$$

For a given machine learning model, the set of all possible hyperparameter configurations is referred to as the search space, denoted by Ω . As described in Fig. 1, each hyperparameter configuration in the search space determines a specific learning model and an algorithm. It also corresponds to its performance value, $U(\Theta|\mathbf{X}_{tr}, \mathbf{X}_{va})$, where $\mathbf{X}_{tr}$ and $\mathbf{X}_{va}$ represent the training and validation data sets, respectively. The procedure for determining the performance value for a hyperparameter configuration, known as performance evaluation, typically requires

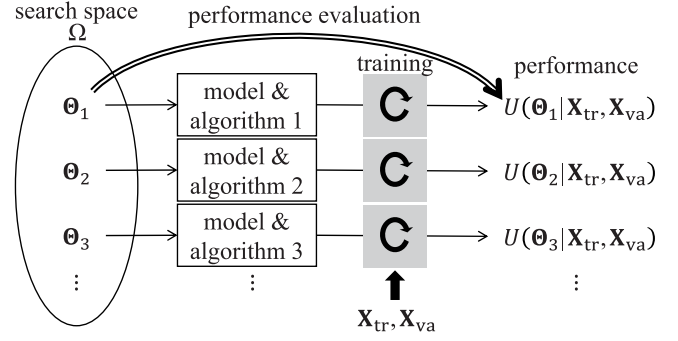


Fig. 1. Relation between hyperparameter configurations and their performances [17].

significant computational time and budget. HPO involves identifying a hyperparameter configuration in the search space that yields the best performance. This process can be represented in a generic mathematical form of an optimization problem [17]:

$$\underset{\Theta}{\text{maximize}} \quad U(\Theta|\mathbf{X}_{tr}, \mathbf{X}_{va}) \quad (1)$$

$$\text{subject to} \quad \Theta \in \Omega. \quad (2)$$

In problem (1), the objective function, $U(\Theta|\mathbf{X}_{tr}, \mathbf{X}_{va})$, represents the performance measure of a machine learning model for a given hyperparameter configuration, Θ , evaluated using the given training and validation data sets, $\mathbf{X}_{tr}$ and $\mathbf{X}_{va}$, respectively. Examples of performance measures include the sum of squared errors (SSEs), mean absolute error (MAE), coefficient of determination (R^2), accuracy, precision, and recall. The objective of HPO is to maximize¹ this performance function by selecting the optimal hyperparameter configuration in the search space.

The decision variable in problem (1) is the hyperparameter configuration, Θ , consisting of n hyperparameters. The nature of these hyperparameters shapes the search space, Ω . Typically, machine learning considers three types of hyperparameters: continuous, discrete, and categorical. Continuous hyperparameters, such as the learning rate for gradient descent, can take values from a continuous range of real numbers. A discrete hyperparameter describes a hyperparameter whose possible values are discrete, such as an integer. The batch size for mini-batch gradient descent is an example of a discrete hyperparameter as its value must be an integer. Continuous and discrete hyperparameters are collectively referred to as numerical hyperparameters because their values are expressed as numbers. Meanwhile, hyperparameters that cannot be represented numerically, such as an optimizer for a learning algorithm, are classified as categorical.

Another important hyperparameter category in machine learning is conditional hyperparameters [29]. These are only presented in a machine learning model when a specific value is assigned to another hyperparameter. For instance, if the optimizer for a learning algorithm is set to stochastic gradient descent (SGD), no momentum coefficient is required [30]. By contrast, if the optimizer is set to momentum, the momentum coefficient is regarded as a hyperparameter [31], which implies it is a conditional hyperparameter.

¹ For performance functions such as SSE and MAE, a smaller value implies better performance. However, we can always define a maximization problem by negating the objective function of the minimization problem [28]. Thus, without the loss of generality, we consider the maximization problem for HPO in this paper.

2.1.2. Difficulties in hyperparameter optimization

To determine the optimal hyperparameter configuration for a machine learning model, we need to solve the HPO problem, as defined in problem (1). However, owing to the intrinsic characteristics of the HPO problem, it is difficult to develop an effective algorithm to solve the HPO problem. Therefore, existing works on HPO have mainly focused on resolving these issues [17]. Accordingly, to analyze the HPO algorithms, it is necessary to first examine the fundamental difficulties in solving the HPO problem itself.

Intractable performance function. A performance function, which is the objective function of the HPO problem, is typically a black-box function with respect to a hyperparameter configuration, Θ . As a result, traditional conventional optimization algorithms cannot usually be applied to solve the HPO problem. Even when the objective function can be explicitly represented in terms of hyperparameter configurations, it is often so complicated that solving the problem becomes impractical.

Search space size. Given the long computational time required for performance evaluations, it is practically impossible to obtain all possible configurations. This is particularly important when the search space contains an infinite number of hyperparameter configurations, rendering it impossible to evaluate all of them. Therefore, selecting a subset of appropriate hyperparameter configurations for evaluation is a critical step in HPO.

Types of hyperparameters. Optimization problems involving discrete or categorical decision variables are particularly challenging because their gradients or derivatives cannot be computed. This limitation rules out the direct use of well-known optimization methods, such as gradient descent and interior-point methods. Moreover, conditional hyperparameters increase complexity to the search space structure, further complicating the problem.

Number of hyperparameters. Machine learning models typically include numerous hyperparameters. As the hyperparameter number for optimization increases, the complexity of the HPO problem increases notably. Further, optimizing such a high-dimensional decision variable results in extremely high computational complexity, rendering traditional optimization methods ineffective.

Computational time for performance evaluation. Recent machine learning models often contain a huge number of learning parameters to update and require large data sets to train their models. These factors considerably extend the computational time for performance evaluations [32].

2.1.3. Basic approaches for hyperparameter optimization

Section 2.1.2 discussed primary challenges in solving the HPO problem. Here, we introduce widely used HPO approaches [18,33–35], which attempt to resolve difficulties in the HPO problem.

Grid search. Given the difficulty in assessing the performance of all hyperparameter configurations in a search space, it is crucial to prudently select hyperparameter configurations for evaluations. *Grid search* [33] offers a simple method for this selection, wherein hyperparameter configurations for evaluations are determined in advance by manually specifying values for each hyperparameter. For continuous hyperparameters, discretization is often applied to choose a finite number of values. Then, performance evaluations are performed for all possible combinations of these values. Fig. 2(a) shows an example of hyperparameter configurations selected in a grid search.

Random search. In grid search, to increase the possibility to find the optimal hyperparameter configuration, more values should be specified for each hyperparameter. However, this dramatically increases the number of performance evaluations, which causes significant inefficiency. *Random search* [34] is an approach designed to overcome this issue. Without prior information about the performance of each hyperparameter configuration, random search assumes all hyperparameter configurations in the search space have an equal chance of being optimal. Then, hyperparameter configurations are chosen uniformly randomly from the search space, as shown in Fig. 2(b). Research

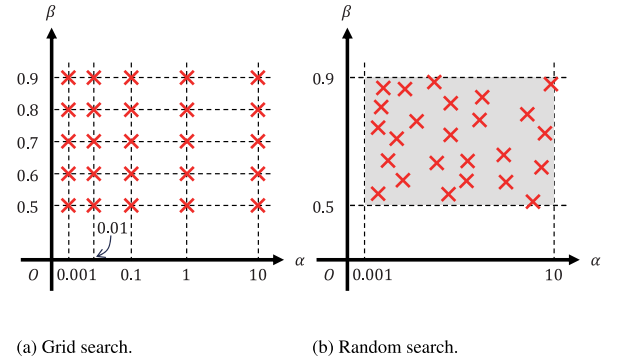


Fig. 2. Examples of grid search and random search.

shows that, for a given number of evaluations, the possibility of finding the optimal hyperparameter configuration is higher than that of grid search. This makes random search particularly more efficient in high-dimensional search spaces.

Evolutionary algorithm. Grid search and random search offer the advantage of evaluating multiple hyperparameter configurations in parallel, since these evaluations are independent of one another. Nevertheless, as there are no specific rules for sampling hyperparameter configurations, their best-performing hyperparameter configurations may deviate significantly from the true optimal solution. To address this, evolutionary algorithms, efficient metaheuristic techniques designed to solve black-box optimization problems, have been employed to improve the effectiveness of hyperparameter sampling. The HPO algorithm introduced in [35] is based on the *Covariance Matrix Adaptation Evolution Strategy (CMA-ES)* [36]. CMA-ES is an evolutionary algorithm that samples potential optimal solutions from a multivariate normal distribution. This distribution is updated at each step according to the performance of samples from previous steps. Using this sophisticated sampling model, the algorithm improves the likelihood of finding a more optimal solution compared to grid search or random search.

BO. If the probability of each hyperparameter configuration being optimal within the search space can be predicted, we can evaluate hyperparameter configurations more effectively than with grid or random search. *BO* [37] provides a novel approach that reflects this motivation for HPO [18]. BO evaluates hyperparameter configurations sequentially, with each new configuration determined based on the results of previous evaluations. A surrogate model is built from these results to predict the performance function across the search space. An acquisition function then uses this surrogate model to estimate the attractiveness of each hyperparameter configuration for subsequent performance evaluation.

The overall procedure for a BO-based HPO method is summarized in Algorithm 1. The algorithm begins with selecting an initial set of n hyperparameter configurations, which can be chosen randomly or via a space-filling method, followed by their performance evaluation. The process of updating the surrogate model, maximizing the acquisition function, and obtaining a new hyperparameter configuration with its performance value is repeated until a given termination condition is met.

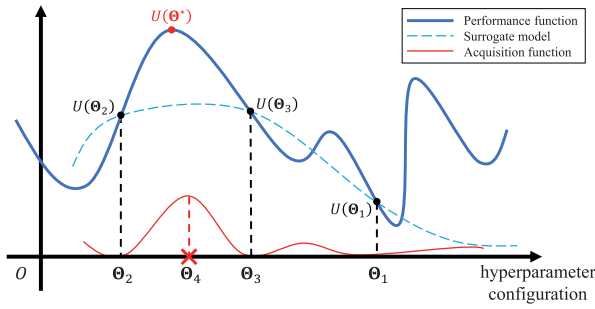
An example of Algorithm 1 is described in Fig. 3, where the objective is to find the optimal hyperparameter configuration, Θ^* , which shows the best performance value, $U(\Theta^*)$, among all configurations in the search space for a given black-box performance function, $U(\Theta)$. The figure displays three hyperparameter configurations, Θ_1 , Θ_2 , and Θ_3 , along with the corresponding performance values, $U(\Theta_1)$, $U(\Theta_2)$, and $U(\Theta_3)$. Using these data, the surrogate model, $f_1(\Theta | \{(\Theta_i, U(\Theta_i))\}_{i=1}^3)$, and its acquisition function, $a_1(\Theta | f_1)$, are obtained and presented. The fourth hyperparameter

Algorithm 1 BO-based HPO

```

input  $\{\Theta_i\}_{i=1}^n$ 
Obtain  $U(\Theta_1), \dots, U(\Theta_n)$  through performance evaluation.
 $t \leftarrow 1$ 
while the termination condition is not met do
  Update a surrogate model  $f_t(\Theta | \{(\Theta_i, U(\Theta_i))\}_{i=1}^{n+t-1})$ .
  Update an acquisition function  $a_t(\Theta | f_t)$ .
   $\Theta_{n+t} \leftarrow \arg \max_{\Theta \in \Omega} a_t(\Theta | f_t)$ 
  Obtain  $U(\Theta_{n+t})$  through performance evaluation.
   $t \leftarrow t + 1$ 
end while
 $\Theta^* \leftarrow \arg \max_{\Theta_i} U(\Theta_i)$ 

```

Fig. 3. Example of BO ($n = 1, t = 3$).

configuration, Θ_4 , which maximizes the acquisition function value, is then selected for another performance evaluation.

In Fig. 3, the surrogate model, f_t , and acquisition function, a_t , serve merely as illustrative examples. Surrogate models and acquisition functions are critical components for constructing a BO algorithm. Commonly used surrogate models include Gaussian processes [38], random forests [39], and Parzen estimators [40]. Widely used acquisition functions include the probability of improvement [41], expected improvement [37], and Gaussian process upper confidence bound (GP-UCB) [42]. Given their inherent advantages and disadvantages, it is imperative to select the appropriate surrogate model and acquisition function for BO. Most studies on BO have focused on modifying and developing surrogate models and acquisition functions to better address particular HPO problems. Further details on the BO-based HPO methods are included in [19,43].

Because all four approaches (grid search, random search, an evolutionary algorithm, and BO) sample a finite number of hyperparameter configurations from the search space to evaluate their performance, they address challenges posed by ‘intractable performance functions’ and ‘search space size.’ Regarding challenges of ‘types of hyperparameters,’ all four approaches are applicable to continuous hyperparameters. While grid search and random search are applicable to discrete, categorical, and conditional hyperparameters, the evolutionary algorithm and BO are not always suited for these hyperparameters. For example, CMA-ES, originally developed for continuous hyperparameters, can be used for discrete hyperparameters with slight modifications. Similarly, while BO using Gaussian processes is generally difficult to apply to discrete and categorical hyperparameters, using random forests or Parzen estimators as surrogate models enables BO to handle these cases. When it comes to the ‘number of hyperparameters,’ random search demonstrates greater resilience to high-dimensional hyperparameter configurations than grid search. The evolutionary algorithm and BO are inefficient for a high-dimensional search space [43,44]. Finally,

all four approaches face the challenge of ‘computational time for performance evaluation,’ a limitation that has driven the development of multi-fidelity HPO methods.

2.2. Multi-fidelity hyperparameter optimization

In this subsection, we introduce the concept of general multi-fidelity optimization and describe its application to HPO. We then briefly analyze basic approaches for multi-fidelity HPO.

2.2.1. Multi-fidelity optimization

Before explaining multi-fidelity HPO, we initially need to introduce the concept of multi-fidelity optimization. We consider the following optimization problem:

$$\underset{x}{\text{maximize}} \quad f(x), \quad (3)$$

where the objective function f is a black-box function, and a significant amount of computational budget is required to measure $f(x)$. To solve problem (3), however, it is necessary to obtain $f(x)$ for a number of x .

This scenario introduces the concept of fidelity, which refers to the accuracy of the obtained value according to the allocated computational budget. The fidelity level i is often modeled as a natural number. The value obtained to approximate $f(x)$ at a fidelity level i is expressed as $f_i(x)$. As the obtained value with higher fidelity is closer to $f(x)$, it is typically assumed that

$$\lim_{i \rightarrow \infty} f_i(x) = f(x), \quad \forall x. \quad (4)$$

In multi-fidelity optimization, both low-fidelity and high-fidelity values are used to make efficient use of the budget, as high-fidelity evaluations for a large number of values for x are too expensive. For example, in problem (3), it will be reasonable to allocate more budget to the values of x where $f(x)$ is expected to be high and less budget to those where $f(x)$ is low. A key aspect of multi-fidelity optimization is assessing potential optimality for each x value. Low-fidelity values, which require less computational budget, can be particularly useful for this purpose. By leveraging these characteristics, multi-fidelity optimization can be applied to HPO, a process referred to as multi-fidelity HPO. In Section 2.2.2, we discuss multi-fidelity HPO in detail.

2.2.2. Multi-fidelity hyperparameter optimization problem

As described in Section 2.1.2, HPO suffers from a long computational time owing to the time-consuming nature of performance evaluations. The characteristics of HPO align well with the multi-fidelity optimization framework described in Section 2.2.1. Thus, incorporating fidelity into performance evaluation can resolve this issue [12].

Fig. 4 illustrates an example of the relation between fidelity levels and performance, a concept often referred to as a learning curve². The figure showcases two learning curves for different hyperparameter configurations, Θ_1 and Θ_2 . The performance value at a low-fidelity level deviates from the actual performance value, and as the fidelity level increases, the performance value converges to the actual value. In this example, at a low-fidelity level with budget 1, the performance of Θ_2 is higher than that of Θ_1 , while at a high-fidelity level with budget B , the performance of Θ_1 is higher than that of Θ_2 . Thus, analyzing the correlation between the low-fidelity and high-fidelity performance values is crucial in multi-fidelity HPO.

In multi-fidelity HPO, it is important to allocate appropriate computational budget to each hyperparameter configuration based on its potential to achieve high performance. Configurations with a higher likelihood of yielding good results should receive a larger share of the budget. Therefore, determining the performance potential of each configuration and appropriate budget allocation is a critical aspect of multi-fidelity HPO. Many studies on multi-fidelity HPO have focused on this aspect.

² A learning curve is a graph that shows the relation between a fidelity level and machine learning performance. Typically, it plots the budget amount on the x-axis and corresponding performance value on the y-axis.

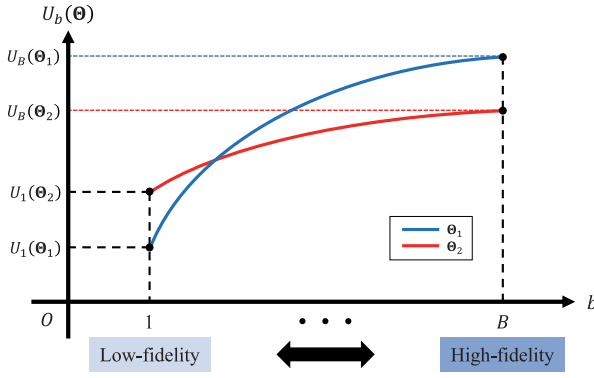


Fig. 4. Example of learning curves.

Algorithm 2 SH-based HPO

Randomly choose $\Theta_1, \dots, \Theta_n$ in Ω .
Initialize b .
Obtain $U_b(\Theta_1), \dots, U_b(\Theta_n)$ through performance evaluation.
while more than one hyperparameter configuration remain **do**
 Remove the half of Θ_i 's that provide low performance.
 $b \leftarrow 2b$
 Obtain $U_b(\Theta_i)$'s for the remaining half of Θ_i 's through performance evaluation.
end while

2.2.3. Basic approaches for multi-fidelity hyperparameter optimization

Among various approaches for applying multi-fidelity optimization to HPO, Successive Halving (SH) is the simplest approach. BO – the most prevalent HPO method – has numerous variants tailored for various multi-fidelity scenarios. Recognizing the advantages of SH and BO for multi-fidelity HPO, it has also been proposed to combine the two approaches. Here, basic approaches for multi-fidelity HPO are introduced.

SH. SH [45] is a simple algorithm that reflects a fundamental principle for multi-fidelity HPO: allocate more budget to hyperparameter configurations with a greater possibility of being optimal. It extends grid and random search by incorporating multi-fidelity optimization. Unlike traditional grid or random search, which allocates a fixed budget to all hyperparameter configurations, SH gradually increases the budget for selected hyperparameter configurations. Algorithm 2 presents the overall procedure for SH. In the first step, the performance is evaluated with a given initial amount, b , of budget for all the given hyperparameter configurations. Subsequently, half of the configurations with the low performance are discarded. The remaining configurations receive an increased budget of $2b$, and their performances are re-evaluated. This process is repeated until a single hyperparameter configuration remains, and it is considered as the optimal one.

A key factor contributing to the short execution time of SH is its parallelization capabilities. As the performance evaluations of hyperparameter configurations in each step are independent, they can be computed in parallel using multiple processors, much like grid or random search. However, it has been demonstrated that SH typically underperforms compared to BO. Consequently, methods combining SH with BO have been proposed, leveraging their respective advantages for HPO, which are introduced later in this paper.

Multi-fidelity BO. The concept of multi-fidelity BO has been developed for various applications, such as shape optimization [46], aerospace design [47], and materials science [48]. These applications involve constructing high-fidelity models by combining surrogate models for different low-fidelity levels, either additively or multiplicatively. This strategy enables the use of multi-fidelity BO [49]. Efforts have

been made to integrate multi-fidelity HPO into BO to reduce its computational time. Because BO comprises two key components, a surrogate model and an acquisition function, numerous studies on multi-fidelity BO have focused on modifying surrogate models or acquisition functions to fit the multi-fidelity setting. When constructing surrogate models, two options are available: using a single multi-fidelity surrogate model or multiple single-fidelity surrogate models for each fidelity level. For example, a Gaussian process that uses the hyperparameter configuration and fidelity level as inputs, called a multi-fidelity Gaussian process, can play the role of a single multi-fidelity surrogate model. By contrast, multiple Gaussian processes can be constructed individually at each fidelity level and can be jointly utilized for multi-fidelity HPO. Comprehensive details on specific approaches for multi-fidelity BO are presented in Section 3.2.

Combining SH with BO. Although BO is a powerful tool to find the optimal solution for HPO, it requires significant computational time owing to its sequential performance evaluation of hyperparameter configurations, which prevents parallel computing. By contrast, SH exhibits rapid convergence but often sacrifices optimality. Thus, efforts have been made to combine SH with BO, capitalizing on their respective advantages. Specifically, BO improves SH by enabling a more sophisticated sampling of hyperparameter configurations. Nevertheless, combining these methods requires careful consideration of several factors—the selection of the surrogate model, design of the SH algorithm, and other implementation details. We introduce some important studies on it in Section 3.3.

3. Algorithms for multi-fidelity hyperparameter optimization

This section provides a detailed introduction to a number of important multi-fidelity HPO algorithms. To provide a comprehensive description, we thoroughly investigate relevant research on multi-fidelity HPO. Specifically, we focus on papers published in leading journals or conferences, those with substantial citation counts, or recent studies in the field. For each algorithm, we describe its motivation and methodology. These algorithms are classified based on approaches introduced in Section 2.2.3: SH, BO, combinations of SH and BO, and other methods. Table 1 summarizes key comparisons among these algorithms.

All algorithms address fundamental difficulties in HPO introduced in Section 2.1.2. Specifically, by employing their own sampling strategies for a finite number of hyperparameter configurations, they resolve the issues of ‘intractable performance functions’ and ‘search space size.’ Furthermore, by employing multi-fidelity optimization, these algorithms address the issue of ‘computational time for performance evaluation.’ However, the ‘types of hyperparameters’ to which each algorithm is applicable differ depending on the specific methodology of each algorithm. Table 1 summarizes the types of hyperparameters that each algorithm can support. Lastly, in terms of the issue of ‘number of hyperparameters,’ while these algorithms are effective for hyperparameter configurations with dimensions between 10 and 20 [43,44], as the dimension increases, their efficiency becomes inadequate for practical implementation due to the curse of dimensionality.

3.1. Successive Halving-based algorithms

Numerous studies have focused on improving multi-fidelity HPO by building on SH. As SH-based algorithms share a simple, straightforward idea to utilize the multi-fidelity concept, they are usually easy to implement. However, they typically possess a relatively high probability to miss the optimal hyperparameter configuration compared to BO-based ones, which requires further improvements to enhance optimality. In this subsection, an overview of multi-fidelity HPO algorithms developed based on SH is provided.

Although parallel computing can accelerate the SH algorithm, the time required to evaluate each hyperparameter configuration varies.

Table 1
Comparison of current studies on multi-fidelity HPO.

Approach	Reference	Algorithm	Types of hyperparameters	Contributions	Year
SH	[50]	AHSA	Cont., Disc., Cate., Cond.	• Asynchronous parallelism	2020
	[51]	PAHSA		• Dynamic decision on the maximum budget	2023
	[52]	Hyperband		• Control of the number of hyperparameter configurations	2018
	[53]	Multi-objective Hyperband		• Multi-objective HPO	2020
	[54]	DEHB		• Combination of Hyperband and differential evolution	2021
	[55]	MORL		• Control on the learning rate	2023
BO	[56]	Freeze-Thaw BO	Cont., Disc.	• Learning curve prediction	2014
	[57]	Multi-task BO		• Fidelity control through multi-task HPO	2013
	[58]	Fabolas		• Multi-fidelity kernel for GP	2017
	[59]	MF-GP-UCB		• Multi-fidelity acquisition function	2019
	[60]	BOCA		• Multiple types of continuous fidelities	2017
	[61]	MF-MES		• Reliable evaluation of the information gain	2020
				• Asynchronous parallelism	
	[62]	DNN-MFBO		• Neural network for a surrogate model	2020
	[63]	DyHPO		• Neural network for a surrogate model	2022
				• Learning curve prediction	
SH + BO	[64]	DPL	Cont., Disc., Cate., Cond.	• Power law function-based learning curve prediction	2023
	[65]	ifBO		• In-context learning for a surrogate model	2024
	[66]	Hyperband + BO		• Combination of Hyperband and BO	2017
	[67]	Hyperband + TPE		• Combination of Hyperband and TPE	2018
	[68]	BOHB		• Combination of Hyperband and TPE	2018
	[69]	MFES-HB		• Employing a large amount of low-fidelity data	2021
Others	[70]	A-BOHB	Cont., Disc., Cate.	• Asynchronous parallelism	2020
	[71]	Hyper-Tune		• Efficient scheduling for parallelism	2022
	[72]	Predictive termination criterion		• Learning curve prediction	2015
	[73]	Dynamic fidelity selection		• Multi-fidelity CMA-ES	2023
	[74]	TSE		• General framework for derivative-free optimization	2019
	[75]	MFPDOO		• Multi-fidelity hierarchical partitioning	2018
	[76]	MFPDOO	Cont.	• Considering noise	2019

TPE: Tree-structured Parzen estimator, Cont.: Continuous hyperparameter, Disc.: Discrete hyperparameter, Cate.: Categorical hyperparameter, Cond.: Conditional hyperparameter.

For example, machine learning models corresponding to different hyperparameter configurations may involve varying numbers of learning parameters. However, hyperparameter configurations whose performances are evaluated earlier must wait for others to finish, leading to significant wastage of time. To resolve this issue, Li et al. [50] introduced an algorithm called *Asynchronous SH Algorithm (ASHA)*. AHSA schedules hyperparameter configurations adaptively across processors. When a processor finishes evaluating a configuration, it immediately begins evaluating the next one as per the scheduling system, rather than waiting for other configurations to complete. This method effectively leverages SH parallelization, reducing the execution time. In [51], ASHA was extended to *Progressive ASHA (PASHA)* to utilize the limited budget more efficiently. Unlike AHSA, PASHA terminates the SH process if the best hyperparameter configuration remains unchanged over two consecutive steps. At this point, the hyperparameter configuration is regarded as the optimal solution, and the total execution time is reduced by terminating the procedure early. PASHA consumes substantially lower budget than ASHA.

The trade-off between n and B/n in SH was tackled in [52], where n is the number of initial hyperparameter configurations to be evaluated and B represents the total amount of available budget for each step. Accordingly, B/n implies the average budget allocated to each hyperparameter configuration in the first step of SH. When the value of n is too large, there is an adequate number of initial hyperparameter

configurations, but then the average budget for each hyperparameter configuration becomes limited. By contrast, when the value of n is too small, each hyperparameter configuration receives sufficient budget, but the total number of configurations evaluated is reduced. Therefore, striking an optimal balance between n and B/n is essential to enhance SH performance. *Hyperband* [52] tackles this issue by introducing flexibility in the initial number of hyperparameter configurations to be evaluated in SH. In addition, Hyperband controls the proportion of hyperparameter configurations η discarded in each step³. Specifically, after each step, only $1/\eta$ of hyperparameter configurations advances to the next round, while the remaining configurations are allocated η times the budget. By considering the number of hyperparameter configurations and setting the appropriate proportion, η , Hyperband enhances SH performance.

In [53], Hyperband [52] was extended to multi-objective HPO. Beyond focusing on model accuracy, HPO in machine learning can target objectives such as fairness, interpretability, and privacy. Schmucker et al. [53] proposed a novel algorithm for multiple objective functions in HPO utilizing Hyperband. In their approach, weighted sums of multiple objective functions are calculated, converting the vector of

³ In the basic SH algorithm, η is set to 2.

objective functions into a single scalar value. This allows for determining the optimal hyperparameter configuration while considering a multi-objective setting.

In Hyperband [52], initial hyperparameter configurations for performance evaluation are sampled using random search. However, this approach can render the performance of the final hyperparameter configuration significantly far from the optimal performance value. Awad et al. [54] addressed this issue, developing *differential evolution and Hyperband (DEHB)*. This approach combines Hyperband [52] with differential evolution [77]. In DEHB, hyperparameter configurations for performance evaluation are selected through differential evolution, which samples configurations in a more sophisticated way. In particular, differential evolution is known for its superiority in high-dimensional spaces. DEHB outperforms Hyperband, particularly when dealing with discrete-valued hyperparameters in a high-dimensional space.

Lee et al. [55] introduced an SH-based algorithm called *Multi-fidelity Optimization with a Recurring Learning rate (MORL)*. This algorithm integrates a learning rate control method into SH, specifically for CNNs. MORL employs a learning rate schedule that decreases over epochs while periodically initializing the learning rate. Specifically, it is re-initialized at the start of each SH step and gradually decreased as the number of epochs increases. Empirical results show that this approach enables CNN models to identify promising solutions more effectively.

3.2. Bayesian optimization-based algorithms

As described in Section 2.1.3, BO is the most-widely used method for HPO. It typically provides an optimal solution from a relatively small number of performance evaluations compared to grid search, random search, and an evolutionary algorithm. However, surrogate models and acquisition functions often require complex designs, often involving sophisticated methodologies. Consequently, significant efforts have been made to incorporate multi-fidelity approaches into BO-based HPO algorithms. Certain notable algorithms are introduced in this subsection.

In [56], *Freeze-Thaw BO* extends BO to the multi-fidelity settings. This approach allocates additional budget to a hyperparameter configuration or halts its performance evaluation based on predictions of its learning curve. Therefore, accurate learning curve predictions are critical to this approach, and are achieved using a Gaussian process as a surrogate model. This Gaussian process takes multiple performance values for one hyperparameter configuration at different fidelity levels as its input, implying that it is a single multi-fidelity Gaussian process. Initially, Freeze-Thaw BO evaluates the performance of numerous hyperparameter configurations over a small number of epochs and then freezes them. It then decides whether to thaw certain configurations (i.e., continue their evaluation for additional epochs) or to evaluate new configurations that have not yet been evaluated, based on the surrogate model's learning curve predictions.

Another notable multi-fidelity BO algorithm is *Multi-task BO*, proposed in [57]. This method adjusts the surrogate model of BO to handle multiple fidelity levels, where each fidelity level of a hyperparameter configuration corresponds to a distinct task. The surrogate model is a Gaussian process with a vector-valued output, where the vector's dimensions correspond to the number of tasks, i.e., fidelity levels. As the Gaussian process reflects the correlation among different fidelity levels, the performance of high-fidelity hyperparameter configurations can be estimated based on low-fidelity ones using this multi-fidelity Gaussian process. Additionally, an acquisition function has been developed that dynamically determines the fidelity level for performance evaluation. By leveraging the proposed surrogate model and acquisition function, multi-task BO can efficiently utilize the budget to identify the optimal hyperparameter configuration.

In [58], multi-task BO was improved. A method called *FAst BO on Large data Sets (Fabolas)* was proposed to accelerate BO for large data

sets using the training data set size as the budget to control fidelity. As the training data set size increases, which implies high fidelity, computational complexity also increases. To address this, Fabolas uses small subsets of the training data set for training machine learning models and extrapolates results to the entire training data set. A surrogate model used in Fabolas is a Gaussian process whose input is a pair of a hyperparameter configuration and a fidelity level. Thus, Fabolas introduces a novel Gaussian process kernel that accounts for its surrogate model, which uses the hyperparameter configuration and training data set size as inputs. This kernel enables Fabolas not only to select hyperparameters to be evaluated but also to identify the smallest training data set size that reliably approximates the full training data set at each step. Furthermore, to make the acquisition function smoother and easier to implement, the budget (i.e., the training data set size in this case) is modeled as a continuous variable.

In [59], GP-UCB, which is one of the representative acquisition functions in BO, was improved to fit the multi-fidelity setting. This enhanced GP-UCB is called *multi-fidelity GP-UCB (MF-GP-UCB)*. To construct MF-GP-UCB, it is first necessary to obtain individual GP-UCB for each fidelity level. MF-GP-UCB is then determined by integrating these GP-UCBs across all fidelity levels and hyperparameter configurations. Once the optimal hyperparameter configuration for performance evaluation is identified, based on the maximum value of MF-GP-UCB, the corresponding fidelity level for its evaluation is selected. [59] demonstrated that typical BO can be extended to the multi-fidelity settings simply by substituting its acquisition function with MF-GP-UCB.

BO with continuous approximations (BOCA), introduced in [60], considers a continuous spectrum of approximations rather than relying on a finite set of low-fidelity performance values to approximate high-fidelity values. BOCA supports multiple types of budgets. For example, if one wishes to estimate the performance of a hyperparameter configuration based on a data set of size N at the T th epoch, a continuous two-dimensional space for fidelity, $[1, N] \times [1, T]$, can be utilized. To this end, BOCA utilizes the key idea of MF-GP-UCB [78]⁴ and extends them to handle multiple types of continuous fidelity levels. By leveraging continuous approximations, BOCA offers great flexibility in approximating high-fidelity values for hyperparameter configurations. This, in turn, can enhance the performance of machine learning models.

Multi-fidelity max-value entropy search (MF-MES), detailed in [61], is a novel information theory-based algorithm used to estimate a reliable high-fidelity hyperparameter configuration using data from low-fidelity levels. MF-MES uses a single multi-fidelity Gaussian process as the surrogate model, with inputs including the hyperparameter configuration, its performance, and its fidelity level. In addition, MF-MES defines an acquisition function based on mutual information between a hyperparameter configuration at a specific fidelity level and the optimal solution identified so far. A high mutual information value implies that the given hyperparameter configuration and fidelity level provide valuable insights into the optimal solution. This acquisition function helps select configurations that hold the most promise for finding the optimal solution. Additionally, in [61], a method for integrating the algorithm with asynchronous parallelization has been proposed, which can effectively reduce the computational time. Consequently, MF-MES can obtain the highly reliable optimal hyperparameter configuration in a relatively short computational time.

In [61], a kernel function of a Gaussian process was modified to fit the multi-fidelity setting. However, these kernel functions were often overly simplistic, failing to adequately reflect the relationships between performances across different fidelity levels. *Deep Neural Network Multi-Fidelity BO (DNN-MFBO)* [62] addresses this issue by employing a neural network, whose input is a hyperparameter configuration, its

⁴ Although [59] has been published after issuing [60], the previous version [78] of [59] has been released before [60]. [78] contains a key idea of MF-GP-UCB.

fidelity level, and its performance. This approach enables a more sophisticated representation of fidelity correlations. The training data set for this neural network is a set of hyperparameter configurations with their fidelity levels and performances. Unlike traditional deep learning, this data set is much smaller; hence, training the neural network for DNN-MFBO is not challenging. In addition, an efficient acquisition function based on [61] to decide the hyperparameter configuration and fidelity has been proposed. The BO-based method using the novel surrogate model and acquisition function effectively captures the complicated correlation among fidelity levels.

Dynamic multi-fidelity HPO (DyHPO) [63] revisits the idea of a surrogate model with a neural network. DyHPO incorporates learning curve predictions into its surrogate model using a Gaussian process with a neural network-based kernel function. Unlike [62], the inputs of the neural network in DyHPO include not only the hyperparameter configuration and current budget but also the learning curve. This design enhances the accuracy of the surrogate model in estimating performance values for hyperparameter configurations. DyHPO also enhances the expected improvement acquisition function to align with the multi-fidelity setting. Studies have demonstrated that DyHPO outperforms existing HPO methods such as random search [34], ASHA [50], and DNN-MFBO [62].

In [64], a surrogate model that predicts high-fidelity performance with great reliability was developed. Although the surrogate model in [63] predicts high-fidelity performance, it relies exclusively on the assumption that performance increases with the fidelity level and converges eventually. Instead, the surrogate model in [64] leverages the widely observed property that learning curves typically follow a power law function, which improves the accuracy of high-fidelity performance predictions. The surrogate model, called a *Deep Power Law (DPL)* ensemble, is an ensemble of neural networks designed to fit a learning curve based on the power law function. By predicting the performance of each hyperparameter configuration at the full budget using the DPL ensemble, the next hyperparameter configuration to be evaluated is effectively determined at each step of the BO algorithm.

Although algorithms proposed in [56,63,64] efficiently utilize the budget by incrementally allocating it to hyperparameter configurations, they face large computational overhead owing to the need to update their surrogate models at each step. Especially, surrogate models in [63,64] contain neural networks, implying that many parameters must be updated at each step. To address this issue, a Freeze-Thaw BO-based algorithm, called *in-context freeze-thaw BO (ifBO)*, has been proposed [65]. The surrogate model in ifBO employs in-context learning [79], which eliminates the need for refitting during algorithm execution. To apply in-context learning to BO, a novel approach to generate prior data to pretrain the surrogate model has been suggested in [65]. As a result, ifBO outperforms existing Freeze-Thaw BO-based algorithms [63,64].

3.3. Successive Halving and Bayesian optimization-based algorithms

SH-based algorithms combined with BO exhibit the strengths of both algorithms, thereby demonstrating their superiority over individual SH- and BO-based algorithms. However, this combination introduces certain disadvantages. For instance, while the original SH-based algorithms are applicable to all types of hyperparameters, combining them with BO can impose constraints on the types of hyperparameters that can be optimized. In addition, parallel implementation becomes more challenging compared to using SH alone. Here, we introduce several algorithms that combine the SH-based algorithms with BO.

Hyperband [52] is known for its relatively short execution time, but its disadvantage lies in the poor performance of the final hyperparameter configuration it may select. By contrast, BO [37] is a well-known HPO algorithm that delivers high performance but requires long computational times. To resolve Hyperband's limitation, i.e., its potential poor performance, it has been combined with BO

in studies [66–68]. Unlike Hyperband, which selects hyperparameter configurations randomly, these combined methods use BO for more informed selection. That is, the role of BO in these methods is to choose the next hyperparameter configuration to evaluate. Nevertheless, each of these methods differs significantly. In [66], a Gaussian process [38] serves as the surrogate model for BO, while [67,68] use a Parzen estimator [40] to efficiently handle multiple hyperparameter configurations. Furthermore, in [67], the surrogate model is built from scratch during each Hyperband step. Conversely, in [68], the surrogate model is updated incrementally, using the model from the previous step. The method proposed in [68], referred to as *BO and Hyperband (BOHB)*, enhances performance compared to the original Hyperband method, exhibiting fast convergence.

Despite its advantages, BOHB [68] fails to fully leverage the combination of Hyperband and BO as it primarily relies on limited high-fidelity performance values, often neglecting the potential of low-fidelity evaluations. To overcome this issue, Li et al. [69] enhanced BOHB by allowing it to utilize not only a small number of high-fidelity hyperparameter configurations but also a large number of low-fidelity ones. To this end, they introduced a multi-fidelity surrogate model called *Multi-Fidelity Ensemble Surrogate (MFES)*. MFES is a weighted sum of multiple surrogate models for each fidelity level. Models representing higher-fidelity levels are assigned greater weights to emphasize their impact over lower-fidelity models. MFES resolves the issue of using limited high-fidelity performance values in BOHB by utilizing low-fidelity ones. The proposed BOHB-based algorithm that utilizes MFES as the surrogate model is called *MFES-HB*.

As mentioned in Section 3.1, an asynchronous algorithm was introduced for SH in [50] for realizing parallel evaluations. However, although BOHB [68] builds on SH, it still employs SH in a synchronous manner. To address this limitation, Klein et al. [70] proposed *asynchronous BOHB (A-BOHB)*, equipping asynchronous Hyperband with BO. As the Parzen estimator – surrogate model in BOHB – poses challenges for asynchronous extension, A-BOHB utilizes a Gaussian process that accounts for the fidelity level of each hyperparameter configuration as its surrogate model. Empirical evaluations reveal that A-BOHB significantly outperforms ASHA [50] and BOHB [68].

Another notable advancement is *Hyper-Tune*, proposed in [71]. Hyper-Tune integrates BO with delayed ASHA (D-ASHA) – a variant of ASHA [50], which offers a more efficient parallel computing schedule for each processor, thereby reducing the total execution time for SH. Similar to BOHB [68], BO is then combined with D-ASHA to form Hyper-Tune. For the surrogate model for BO, MFES [69] is utilized with a novel method to determine the weight for each fidelity level appropriately. Hyper-Tune outperforms existing HPO methods, such as BOHB [68] and A-BOHB [70].

3.4. Other approaches

Although numerous studies on HPO are based on SH or BO, others explore multi-fidelity HPO without relying on these methods. This subsection highlights several works on multi-fidelity HPO that do not employ SH or BO.

In [72], learning curve prediction models are applied to multi-fidelity HPO, similar to approaches in [56,63]. This approach introduces various learning curve prediction models, which are then combined with ensemble models. These ensembles are the weighted sums of learning curve prediction models, calculated using a given set of weights. Based on these weighted sums, a hyperparameter configuration continues to be evaluated only if its predicted performance exceeds the best recorded value. Otherwise, performance evaluation is terminated, and a new hyperparameter configuration is tested. This procedure, called the *predictive termination criterion*, minimizes the time spent on evaluating configurations unlikely to yield optimal results. In this case, identifying an accurate model for the learning curve is critical, as this ensures the precise predictions of a configurations's final

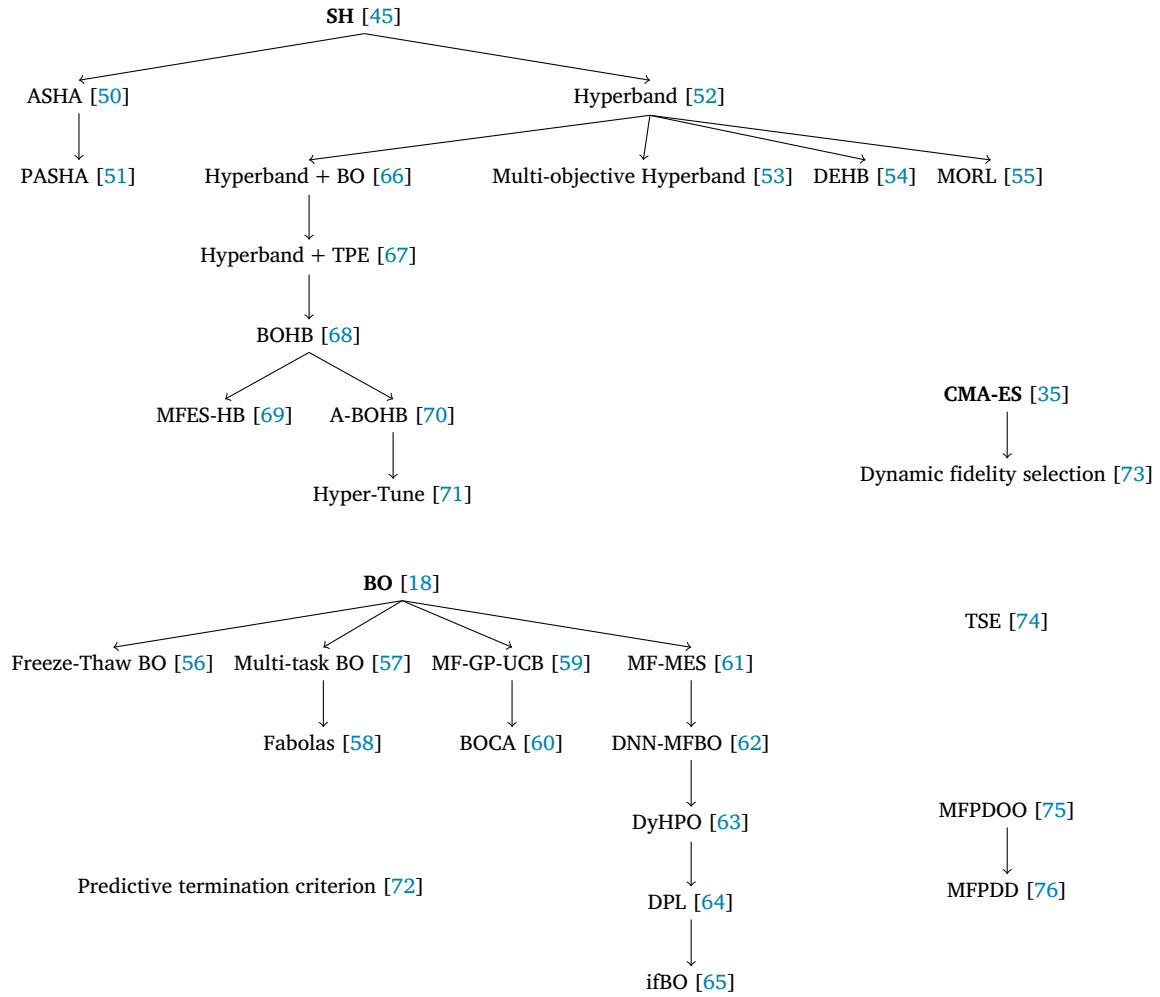


Fig. 5. Relation among the multi-fidelity HPO algorithms.

performance. While this method is employed to BO in [72], it can be employed to any other HPO algorithm, such as grid/random search, CMA-ES, and SH, enabling the inclusion of multi-fidelity concepts to any HPO algorithm.

Another widely used approach for HPO is based on evolutionary algorithms, such as CMA-ES [35]. In [73], CMA-ES was adapted to enable multi-fidelity HPO. While performance evaluation is conducted using a fixed budget for all hyperparameter configurations in conventional CMA-ES, a dynamic fidelity selection strategy was proposed in [73]. The fidelity level gradually increases at each step of the algorithm. The process terminates when the best hyperparameter configuration at the current step is deemed likely to remain optimal, as determined using the Kendall's rank correlation coefficient [80]. This strategy demonstrates that an evolutionary HPO algorithm can be extended to the multi-fidelity variant.

Hu et al. [74] proposed a novel multi-fidelity HPO framework, designed to address challenges such as the non-differentiability, non-continuity, and non-convexity of performance functions, making it broadly applicable to any derivative-free function. The framework introduces a predictor that estimates actual performance values by combining numerous low-fidelity evaluations with a small set of high-fidelity performance ones. However, the limited number of high-fidelity performance values initially leads to lower reliability of the predictor for these values. This issue is resolved utilizing the *Transfer Series Expansion (TSE)* method, which employs a linear combination of multiple base predictors. Through this method, the predictor for high-fidelity performance can be effectively constructed, enabling multi-fidelity HPO in the proposed HPO framework.

A novel HPO algorithm was introduced in [81], employing a hierarchical partitioning of the search space to identify the optimal hyperparameter configuration as the step progresses. Then, the multi-fidelity setting is incorporated in it [75,76]. The study in [75] proposed the *Multi-Fidelity Parallel Deterministic Optimistic Optimization (MFPDOO)* algorithm. Here, the search space is further partitioned as the fidelity level increases, refining search for the best configuration. Building on this, [76] extended the method by considering the impact of noise. The corresponding algorithm is referred to as *Multi-Fidelity Parallel Optimistic Optimization (MFPOO)*. In practical scenarios, the performance evaluation of hyperparameter configurations can be affected by noise, resulting in inaccurate measurements. Noise is appropriately modeled and applied to the hierarchical partitioning-based algorithm in [76].

3.5. Discussion

In preceding subsections, we review existing multi-fidelity HPO algorithms. Most of these algorithms are developed by addressing the limitations of other existing algorithms or building upon their concepts. Thus, to better understand them, it is necessary to identify algorithms from which they were derived and how they were modified. To this end, we organize the relationship among algorithms and it is illustrated in 5. For example, DEHB [54] is developed by applying differential evolution to Hyperband [52]. Similarly, BOCA [60] extends the discrete fidelity space of MF-GP-UCB [59] into a continuous one.

As these algorithms often build on the principles of existing methods, they typically demonstrate superior performance than existing

Table 2
Summary of experiments on multi-fidelity HPO algorithms.

Algorithm	Comparisons	Experiment		
		Machine learning		Test function
		Model	Data set	
[50]	[34,45,52,68]	CNN	CIFAR-10	–
	[45,68]	RNN	Penn Treebank	
		LSTM	Penn Treebank	
[51]	[50]	–	–	PD1, NAS-Bench-201
[52]	[45]	LeNet	MNIST	–
	[18,34,39,40,45]	CNN	CIFAR-10, MRBI, SVHN	
[53]	[18,34]	XGBoost	Adult, COMPAS	–
		Neural network	Adult, COMPAS	
[54]	[34,68]	Neural network	Adult, ...	NAS-Bench-101, NAS-Bench-1Shot1, NAS-Bench-201, NAS-HPO-Bench
		Bayesian neural network	Boston Housing, Protein Structure	
		PPO	Cartpole swing-up	
[55]	[34,45,52]	VGG-11	CIFAR-10, CIFAR-100, Tiny ImageNet	–
		AlexNet, ResNet-20, ResNet-56, VGG-16	CIFAR-100	
		VGG-16 (TL)	CIFAR-10, CIFAR-100, Tiny ImageNet	
		CNN (SSL)	CIFAR-100	
[56]	[18]	Logistic regression	MNIST	–
		Online LDA	Wikipedia	
		PMF	MovieLens 100K	
[57]	[18]	Logistic regression	MNIST	–
		CNN	SVHN, STL-10	
		PMF	MovieLens 100K	
		Online LDA	200,000 documents	
[58]	[18,52,57]	SVM	MNIST, ...	–
	[18,57]	CNN	CIFAR-10, SVHN	
		ResNet	CIFAR-10	
[59]	[18]	SVM	MAGIC gamma telescope	–
		SALSA	Coal power plant data set	
		OpenCV	Viola & Jones cascade face	
[60]	[18,59]	SVM	20 news group data set	–
[61]	[60]	–	–	–
[62]	[59,61]	–	–	–
[63]	[34,50,52,54,60,62,68]	–	–	LCBench, TaskSet, NAS-Bench-201
[64]	[34,39,50,52,54,60,68]	–	–	LCBench, PD1, TaskSet, nanoGPT
[65]	[34,50,52,63,64]	–	–	LCBench, PD1, TaskSet

(continued on next page)

ones. Previous studies have compared their proposed algorithm with other existing ones through various experiments (Table 2). In each study, the results generally demonstrate that the algorithm proposed in them outperforms compared ones. We organize the proposed algorithm and its corresponding comparison algorithms in each reference in Table 2. Three methods are employed to evaluate these algorithms: machine learning experiments, benchmark experiments, and test functions. Machine learning experiments involve training models for given data sets and observing their performances. Machine learning models and data sets used in these tests are outlined in Table 2. Instead of directly training models, we can utilize benchmarks for experiments. A benchmark relies on prerecorded data that represent the performance of specific hyperparameter configurations and the associated fidelity levels for a specific machine learning model and data set. These prerecorded performance values are treated as if machine learning experiments are conducted in real time. Finally, an alternative to machine learning is

the utilization of test functions. These are artificial and designed to simulate algorithms under simple scenarios. Please refer to references for details on experiments and their results.

4. Future research directions

Although numerous studies have been conducted on multi-fidelity HPO, as described in Section 3, there are several aspects that need to be further explored. In this section, we propose potential research directions to further improve multi-fidelity HPO.

4.1. Estimation of uncertainty in low-fidelity performance

As previously stated, low-fidelity performance is less reliable and accurate compared to high-fidelity performance. An additional issue in multi-fidelity HPO lies in the uncertainty of low-fidelity values. This

Table 2 (continued).

Algorithm	Comparisons	Experiment		
		Machine learning		Test function
		Model	Data set	
[66]	[18,52]	Keras	CIFAR-10	–
[67]	[34,40,52]	LeNet	MNIST	–
		AlexNet	CIFAR-10, MRBI, SVHN	
		SSD300	PASCAL VOC	
[68]	[18,34,52,58]	SVM	MNIST	–
	[18,34,40,52]	Neural network	MNIST, Adult, Higgs, Letter, Optdigits, Poker	
	[34,40,52]	Bayesian neural network	Boston Housing, Protein Structure	
		PPO	Cartpole swing-up	
	[82,83]	ResNet	CIFAR-10	
[69]	[39,52,58,68,74]	FCNet	MNIST	–
		ResNet	CIFAR-10	
		XGBoost	Coverttype	
		Auto-sklearn	MNIST, Higgs, Letter, Electricity, Krypt, Mv, Poker, Fried, A9a, 2dplanes	
[70]	[50,52,68]	Neural network	OpenML	NAS-Bench-201
		LSTM	WikiText-2	
		ResNet	CIFAR-10	
[71]	[45,50,52,68–70]	XGBoost	Higgs, Pokerhand, Coverttype, Hepmass	NAS-Bench-201
		ResNet	CIFAR-10	
		LSTM	Penn Treebank	
[72]	[39,40]	Neural network	CIFAR-10, MNIST	–
		CNN	CIFAR-10, CIFAR-100	
[73]	[35]	CNN	CIFAR-10	–
[74]	[84,85]	Light-GBM	Musk, HYRU2, Magic04, Adult, Sensorless, Connect, Higgs, Credit, Miniboone, Airline, MovieLens, Criteo	–
[75]	[18,59,60]	SVM	20 news group data set	–
[76]	[18,59,60]	XGBoost	MNIST, ...	–
		SVM	20 news group data set	
		CNN	CIFAR-10	

RNN: Recurrent neural network, LSTM: Long short-term memory, XGBoost: Extreme Gradient Boosting, PPO: Proximal policy optimization, TL: Transfer learning, SSL: Semi-supervised learning, LDA : Latent Dirichlet allocation, PMF: Probabilistic Matrix Factorization, SVM: Support vector machine, SALSA: Stochastic approach for link-structure analysis.

uncertainty is influenced by the fidelity level, hyperparameter types, budget considerations, and others. If the uncertainty levels of different low-fidelity values are known, these values can be utilized individually based on their respective uncertainty levels to predict the true high-fidelity values [86]. While several studies on multi-fidelity HPO have indirectly considered the uncertainty level of different low-fidelity values [87,88], they have not explicitly quantified them. To enhance the performance of multi-fidelity HPO, quantifying the reliability or uncertainty level of low-fidelity values is essential.

4.2. Multi-fidelity hyperparameter optimization with finite budget

In real-world scenarios, the total available computational budget for performance evaluation is often limited. Especially, in applications where the available time or computing budget is scarce, it is critical for an HPO algorithm to consider a finite budget when determining the optimal hyperparameter configuration. For example, Internet of Things devices [89] in a sensor platform typically have a small budget for machine learning tasks. To identify the optimal hyperparameter configuration within these budget constraints, it is necessary to strategically allocate the appropriate amount of the budget across hyperparameter configurations. Several studies have already been conducted on HPO with a finite budget [90,91]. In the multi-fidelity setting, however,

budget allocation is more challenging because the budget affects performance differently depending on various fidelity levels. Techniques such as rollout [92] and learning curve prediction [93] are promising to handle finite budgets in multi-fidelity HPO. Because multi-fidelity HPO is highly related to budget allocation, a limited amount of the budget needs to be considered in studies on multi-fidelity HPO.

4.3. Multi-fidelity hyperparameter optimization with non-stationary data sets

In machine learning, it is often assumed that the training and validation data sets remain fixed throughout the learning process. However, in numerous real-world scenarios, data sets constantly change over time. Examples of such practical applications include autonomous driving [94] and smart healthcare [5]. In such cases, machine learning predicts outputs while the data sets undergo continuous change. When data sets change during learning, the corresponding performance values also shift. This presents challenges in developing algorithms that can adapt to non-stationary data sets. Although numerous studies on machine learning have considered non-stationary data sets [95,96], only few have explored their impact on HPO [97]. Furthermore, considering multi-fidelity in HPO adds further complexity when dealing with non-stationary data sets. These dynamic data sets substantially complicate

the multi-fidelity HPO problem. Thus, advancing multi-fidelity HPO methods by addressing non-stationary data sets is a critical area for further research and development. Utilizing existing methods, such as continual learning [98] and incremental learning [99], can be a solution to multi-fidelity HPO.

5. Conclusion

Performance evaluation in HPO is often highly time-consuming. To address this issue, leveraging low-fidelity values, which can be derived using a small budget but with low accuracy, has emerged as an efficient approach. Consequently, numerous studies have incorporated the multi-fidelity concept into HPO. In this review, we thoroughly analyze studies on multi-fidelity HPO and propose promising directions for future research on multi-fidelity HPO.

CRedit authorship contribution statement

Jonghyeon Won: Writing – original draft. **Hyun-Suk Lee:** Writing – review & editing. **Jang-Won Lee:** Writing – review & editing.

Declaration of competing interest

The authors declare that there is no conflict of interest in this paper.

Acknowledgments

This work was supported by the Technology Innovation Program (RS-2022-00154678, Development of Intelligent Sensor Platform Technology for Connected Sensor) funded By the Ministry of Trade, Industry & Energy(MOTIE, Korea).

References

- [1] P.P. Shinde, S. Shah, A review of machine learning and deep learning applications, in: *Proc. Int. Conf. Comput. Commun. Control Autom.*, 2018, pp. 1–6.
- [2] J. Kim, D.S. Han, Deep learning-based channel quality indicators prediction for vehicular communication, *ICT Express* 9 (1) (2023) 116–121.
- [3] J. Kim, J. Oh, E.-R. Jeong, J. Joung, Convolutional neural network-based transmit antenna selection for UAV-ground station communications with time-varying channels, *ICT Express* 10 (1) (2024) 90–96.
- [4] D. Lim, J. Kim, H. Kim, Efficient robot tracking system using single-image-based object detection and position estimation, *ICT Express* 10 (1) (2024) 125–131.
- [5] A. Safiya Parvin, B. Saleena, Analysis of machine learning and deep learning prediction models for sepsis and neonatal sepsis: A systematic review, *ICT Express* 9 (6) (2023) 1215–1225.
- [6] L.L. Janneh, Y. Zhang, M. Hydar, Z. Cui, Deep learning-based hybrid feature selection for the semantic segmentation of crops and weeds, *ICT Express* 10 (1) (2024) 118–124.
- [7] L. Yang, A. Shami, On hyperparameter optimization of machine learning algorithms: Theory and practice, *Neurocomputing* 415 (2020) 295–316.
- [8] T. Yu, H. Zhu, Hyper-parameter optimization: A review of algorithms and applications, 2020, arXiv:2003.05689.
- [9] F. Hutter, L. Kotthoff, J. Banschoren, *Automated Machine Learning*, Springer Nature, 2019.
- [10] R. Andonie, Hyperparameter optimization in learning systems, *J. Membr. Comput.* 1 (4) (2019) 279–291.
- [11] B. Bischl, M. Binder, M. Lang, T. Pielok, J. Richter, S. Coors, J. Thomas, T. Ullmann, M. Becker, A.-L. Boulesteix, D. Deng, M. Lindauer, Hyperparameter optimization: Foundations, algorithms, best practices, and open challenges, *Wiley Interdiscip. Rev.: Data Min. Knowl. Discov.* 13 (2) (2023) 1–43.
- [12] K. Li, F. Li, Multi-fidelity methods for optimization: A survey, 2024, arXiv:2402.09638.
- [13] D. Huang, T.T. Allen, W.I. Notz, R.A. Miller, Sequential kriging optimization using multiple-fidelity evaluations, *Struct. Multidiscip. Optim.* 32 (2006) 369–382.
- [14] A.I.J. Forrester, A. Söbester, A.J. Keane, Multi-fidelity optimization via surrogate modelling, *Proc. R. Soc. A* 463 (2007) 3251–3269.
- [15] M.G. Fernández-Godino, Review of multi-fidelity models, *Adv. Comput. Sci. Eng.* 1 (2023) 351–400.
- [16] F. Hutter, J. Lücke, L. Schmidt-Thieme, Beyond manual tuning of hyperparameters, *KI- Künstliche Intell.* 29 (2015) 329–337.
- [17] J. Won, J. Shin, J.-H. Kim, J.-W. Lee, A survey on hyperparameter optimization in machine learning, *J. Korean Inst. Commun. Inf. Sci.* 48 (6) (2023) 733–747.
- [18] J. Snoek, H. Larochelle, R.P. Adams, Practical bayesian optimization of machine learning algorithms, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2012, pp. 1–9.
- [19] J. Wu, X.-Y. Chen, H. Zhang, L.-D. Xiong, H. Lei, S.-H. Deng, Hyperparameter optimization for machine learning models based on Bayesian optimization, *J. Electron. Sci. Technol.* 17 (1) (2019) 26–40.
- [20] A. Morales-Hernández, I.V. Nieuwenhuys, S.R. Gonzalez, A survey on multi-objective hyperparameter optimization algorithms for machine learning, *Artif. Intell. Rev.* 56 (2023) 8043–8093.
- [21] F. Karl, T. Pielok, J. Moosbauer, F. Pfisterer, S. Coors, M. Binder, L. Schneider, J. Thomas, J. Richter, M. Lang, E.C. Garrido-Merchán, J. Branke, B. Bischl, Multi-objective hyperparameter optimization in machine learning—An overview, *ACM Trans. Evol. Learn. Optim.* 3 (4) (2023) 1–50.
- [22] J. Mun, S. Ha, J. Lee, DE-DARTS: Neural architecture search with dynamic exploration, *ICT Express* 9 (3) (2023) 379–384.
- [23] T. Elsken, J.H. Metzen, F. Hutter, Neural architecture search: A survey, *J. Mach. Learn. Res.* 20 (55) (2019) 1–21.
- [24] M. Wistuba, A. Rawat, T. Pedapati, A survey on neural architecture search, 2019, arXiv:1905.01392.
- [25] P. Ren, Y. Xiao, X. Chang, P.-Y. Huang, Z. Li, X. Chen, X. Wang, A comprehensive survey of neural architecture search: Challenges and solutions, *ACM Comput. Surv.* 54 (4) (2021) 1–34.
- [26] Y. Liu, Y. Sun, B. Xue, M. Zhang, G.G. Yen, K.C. Tan, A survey on evolutionary neural architecture search, *IEEE Trans. Neural Netw. Learn. Syst.* 34 (2) (2023) 550–570.
- [27] M.-T. Wu, C.-W. Tsai, Training-free neural architecture search: A review, *ICT Express* 10 (1) (2024) 213–231.
- [28] E.K.P. Chong, S.H. Žak, *An introduction to optimization*, John Wiley & Sons, 2013.
- [29] J.-C. Lévesque, A. Durand, C. Gagné, R. Sabourin, Bayesian optimization for conditional hyperparameter spaces, in: *Proc. Int. Joint Conf. Neural Netw.*, 2017, pp. 286–293.
- [30] S.-I. Amari, Backpropagation and stochastic gradient descent method, *Neurocomputing* 5 (4) (1993) 185–196.
- [31] I. Sutskever, J. Martens, G. Dahl, G. Hinton, On the importance of initialization and momentum in deep learning, in: *Proc. Int. Conf. Machine Learn.*, 2013, pp. 1139–1147.
- [32] J. Shin, J. Won, H.-S. Lee, J.-W. Lee, A review on label cleaning techniques for learning with noisy labels, *ICT Express* 10 (6) (2024) 1315–1330.
- [33] G. Montavon, G.B. Orr, K.-R. Müller, *Neural networks: Tricks of the trade*, Springer, 2012.
- [34] J. Bergstra, Y. Bengio, Random search for hyper-parameter optimization, *J. Mach. Learn. Res.* 13 (2012) 281–305.
- [35] I. Loshchilov, F. Hutter, CMA-ES for hyperparameter optimization of deep neural networks, 2016, arXiv:1604.07269.
- [36] N. Hansen, A. Ostermeier, Completely derandomized self-adaptation in evolution strategies, *Evol. Comput.* 9 (2) (2001) 159–195.
- [37] J. Mockus, V. Tiesis, A. Zilinskas, The application of Bayesian methods for seeking the extreme, *Toward. Glob. Optim.* 2 (1978) 117–129.
- [38] C.E. Rasmussen, C. Williams, *Gaussian process for machine learning*, MIT Press, 2006.
- [39] F. Hutter, H.H. Hoos, K. Leyton-Brown, Sequential model-based optimization for general algorithm configuration, in: *Proc. Learn. Intell. Optim.*, 2011, pp. 507–523.
- [40] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, Algorithms for hyper-parameter optimization, *Adv. Neural Inf. Process. Syst.* 24 (2011) 1–9.
- [41] H. Kushner, A new method of locating the maximum point of an arbitrary multipeak curve in the presence of noise, *J. Basic Eng.* 86 (1) (1964) 97–106.
- [42] N. Srinivas, A. Krause, S.M. Kakade, M. Seeger, Gaussian process optimization in the bandit setting: No regret and experimental design, in: *Proc. Int. Conf. Machine Learn.*, 2010, pp. 1015–1022.
- [43] P.I. Frazier, A tutorial on Bayesian optimization, 2018, arXiv:1807.02811.
- [44] M.N. Omidvar, X. Li, A comparative study of CMA-ES on large scale global optimisation, in: *Proc. Australasian Joint Conf. Artif. Intell.*, 2010, pp. 303–312.
- [45] K. Jamieson, A. Talwalkar, Non-stochastic best arm identification and hyperparameter optimization, in: *Proc. Int. Conf. Artif. Intell. Stat.*, 2016, pp. 240–248.
- [46] B. Do, R. Zhang, Multi-fidelity Bayesian optimization in engineering design, 2024, arXiv:2311.13050.
- [47] J. Kim, M. Li, Y. Li, A. Gómez, O. Hinder, P.W. Leu, Multi-BOWs: Multi-fidelity multi-objective Bayesian optimization with warm starts for nanophotonic structure design, *Digit. Discov.* 3 (2) (2024) 381–391.
- [48] V.S. Gil, R. Barbano, D.P. Gutiérrez, J.S. Luterbacher, J.M. Hernández-Lobato, P. Schwaller, L. Roch, Best practices for multi-fidelity Bayesian optimization in materials and molecular research, in: *Neural Inf. Process. Syst. Workshop AI New Drug Modalities*, 2024, pp. 1–11.
- [49] P. Mikkola, J. Martinelli, L. Filstroff, S. Kaski, Multi-fidelity Bayesian optimization with unreliable information sources, in: *Proc. Int. Conf. Artif. Intell. Stat.*, 2023, pp. 7425–7454.

- [50] L. Li, K. Jamieson, A. Rostamizadeh, E. Gonina, J. Ben-Tzur, M. Hardt, B. Recht, A. Talwalkar, A system for massively parallel hyperparameter tuning, *Proc. Mach. Learn. Syst.* 2 (2020) 230–246.
- [51] O. Bohdal, L. Balles, M. Wistuba, B. Ermis, C. Archambeau, G. Zappella, PASHA: Efficient HPO and NAS with progressive resource allocation, in: *Proc. Int. Conf. Learn. Representations*, 2023, pp. 1–20.
- [52] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, A. Talwalkar, Hyperband: A novel bandit-based approach to hyperparameter optimization, *J. Mach. Learn. Res.* 18 (2018) 6765–6816.
- [53] R. Schmucker, M. Donini, V. Perrone, C. Archambeau, Multi-objective multi-fidelity hyperparameter optimization with application to fairness, in: *Proc. Neural Inf. Process. Syst. Workshop Meta-Learning*, 2020, pp. 1–14.
- [54] N.H. Awad, N. Mallik, F. Hutter, DEHB: Evolutionary hyperband for scalable, robust and efficient hyperparameter optimization, in: *Proc. Int. Joint Conf. Artif. Intell.*, 2021, pp. 2147–2153.
- [55] H. Lee, G. Lee, J. Kim, S. Cho, D. Kim, D. Yoo, Improving multi-fidelity optimization with a recurring learning rate for hyperparameter tuning, in: *Proc. IEEE/CVF Winter Conf. Appl. Comput. Vis.*, 2023, pp. 2309–2318.
- [56] K. Swersky, J. Snoek, R.P. Adams, Freeze-thaw Bayesian optimization, 2014, arXiv:1406.3896.
- [57] K. Swersky, J. Snoek, R.P. Adams, Multi-task Bayesian optimization, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 1–9.
- [58] A. Klein, S. Falkner, S. Bartels, P. Hennig, F. Hutter, Fast Bayesian optimization of machine learning hyperparameters on large datasets, in: *Proc. Int. Conf. Artif. Intell. Stat.*, 2017, pp. 528–536.
- [59] K. Kandasamy, G. Dasarathy, J. Oliva, J. Schneider, B. Póczos, Multi-fidelity Gaussian process bandit optimisation, *J. Artificial Intelligence Res.* 66 (2019) 151–196.
- [60] K. Kandasamy, G. Dasarathy, J. Schneider, B. Póczos, Multi-fidelity Bayesian optimisation with continuous approximations, in: *Proc. Int. Conf. Mach. Learn.*, 2017, pp. 1799–1808.
- [61] S. Takeno, H. Fukuoka, Y. Tsukada, T. Koyama, M. Shiga, I. Takeuchi, M. Karasuyama, Multi-fidelity Bayesian optimization with max-value entropy search and its parallelization, in: *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 9334–9345.
- [62] S. Li, W. Xing, R. Kirby, S. Zhe, Multi-fidelity Bayesian optimization via deep neural networks, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2020, pp. 8521–8531.
- [63] M. Wistuba, A. Kadra, J. Grabocka, Supervising the multi-fidelity race of hyperparameter configurations, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 13470–13484.
- [64] A. Kadra, M. Janowski, M. Wistuba, J. Grabocka, Scaling laws for hyperparameter optimization, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 47527–47553.
- [65] H. Rakotoarison, S. Adriaensen, N. Mallik, S. Garibov, E. Bergman, F. Hutter, In-context freeze-thaw Bayesian optimization for hyperparameter optimization, in: *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 1–12.
- [66] H. Bertrand, R. Ardon, M. Perrot, I. Bloch, Hyperparameter optimization of deep neural networks: Combining hyperband with Bayesian model selection, in: *Proc. Conférence sur L'Apprentissage Automatique*, 2017, pp. 1–5.
- [67] J. Wang, J. Xu, X. Wang, Combination of hyperband and Bayesian optimization for hyperparameter optimization in deep learning, 2018, arXiv:1801.01596.
- [68] S. Falkner, A. Klein, F. Hutter, BOHB: Robust and efficient hyperparameter optimization at scale, in: *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1437–1446.
- [69] Y. Li, Y. Shen, J. Jiang, J. Gao, C. Zhang, B. Cui, MFES-HB: Efficient Hyperband with multi-fidelity quality measurements, in: *Proc. AAAI Conf. Artif. Intell.*, (10) 2021, pp. 8491–8500.
- [70] A. Klein, L.C. Tiao, T. Lienart, C. Archambeau, M. Seeger, Model-based asynchronous hyperparameter and neural architecture search, 2020, arXiv:2003.10865.
- [71] Y. Li, Y. Shen, H. Jiang, W. Zhang, J. Li, J. Liu, C. Zhang, B. Cui, Hyper-Tune: Towards efficient hyper-parameter tuning at scale, in: *Proc. Int. Conf. Very Large Data Bases Endowment*, 2022.
- [72] T. Domhan, J.T. Springenberg, F. Hutter, Speeding up automatic hyperparameter optimization of deep neural networks by extrapolation of learning curves, in: *Proc. Int. Joint Conf. Artif. Intell.*, 2015.
- [73] S. Takenaga, Y. Ozaki, M. Onishi, Dynamic fidelity selection for hyperparameter optimization, in: *Proc. Companion Conf. Genetic Evol. Comput.*, 2023, pp. 2304–2307.
- [74] Y.-Q. Hu, Y. Yu, W.-W. Tu, Q. Yang, Y. Chen, W. Dai, Multi-fidelity automatic hyper-parameter tuning via transfer series expansion, in: *Proc. AAAI Conf. Artif. Intell.*, (1) 2019, pp. 3846–3853.
- [75] R. Sen, K. Kandasamy, S. Shakkottai, Multi-fidelity black-box optimization with hierarchical partitions, in: *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 4538–4547.
- [76] R. Sen, K. Kandasamy, S. Shakkottai, Noisy blackbox optimization using multi-fidelity queries: A tree search approach, in: *Proc. Int. Conf. Artif. Intell. Stat.*, 2019, pp. 2096–2105.
- [77] R. Storn, K. Price, Differential evolution—a simple and efficient heuristic for global optimization over continuous spaces, *J. Global Optim.* 11 (4) (1997) 341–359.
- [78] K. Kandasamy, G. Dasarathy, J.B. Oliva, J. Schneider, B. Póczos, Gaussian process bandit optimisation with multi-fidelity evaluations, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [79] Q. Dong, L. Li, D. Dai, C. Zheng, J. Ma, R. Li, H. Xia, J. Xu, Z. Wu, B. Chang, X. Sun, L. Li, Z. Sui, A survey on in-context learning, in: *Proc. Conf. Empir. Methods Nat. Lang. Process.*, 2024, pp. 1107–1128.
- [80] M.G. Kendall, J.D. Gibbons, Rank correlation methods (5th ed.), Oxford University Press, 1990.
- [81] J.-B. Grill, M. Valko, R. Munos, Black-box optimization of noisy functions with unknown smoothness, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2015, pp. 1–9.
- [82] B. Zoph, Q.V. Le, Neural architecture search with reinforcement learning, in: *Proc. Int. Conf. Learn. Representation*, 2017, pp. 1–14.
- [83] C. Liu, B. Zoph, M. Neumann, J. Shlens, W. Hua, L.-J. Li, L. Fei-Fei, A. Yuille, J. Huang, K. Murphy, Progressive neural architecture search, in: *Proc. European Conf. Comput. Vis.*, 2018, pp. 1–16.
- [84] Y. Yu, H. Qian, Y.-Q. Hu, Derivative-free optimization via classification, in: *Proc. AAAI Conf. Artif. Intell.*, 2016, pp. 2286–2292.
- [85] Y.-Q. Hu, H. Qian, Y. Yu, Sequential classification-based optimization for direct policy search, in: *Proc. AAAI Conf. Artif. Intell.*, 2017, pp. 2029–2035.
- [86] B. Peherstorfer, K. Willcox, M. Gunzburger, Survey of multifidelity methods in uncertainty propagation, inference, and optimization, *SIAM Rev.* 60 (3) (2018) 550–591.
- [87] J. Liu, F. Zhang, J. Guan, X. Shen, UQ-guided hyperparameter optimization for iterative learners, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 1–12.
- [88] Y. Zhang, H. Wu, Y. Yang, FlexHB: A more efficient and flexible framework for hyperparameter optimization, 2024, arXiv:2402.13641.
- [89] S. He, K. Shi, C. Liu, B. Guo, J. Chen, Z. Shi, Collaborative sensing in Internet of Things: A comprehensive survey, *IEEE Commun. Surv. Tuts.* 24 (3) (2022) 1435–1474.
- [90] R. Lam, K. Willcox, D.H. Wolpert, Bayesian optimization with a finite budget: An approximate dynamic programming approach, in: *Proc. Adv. Neural Inf. Process. Syst.*, 2016, pp. 1–9.
- [91] Z. Lu, C.-K. Chiang, F. Sha, Hyper-parameter tuning under a budget constraint, 2019, arXiv:1902.00532.
- [92] E. Lee, D. Eriksson, D. Bindel, B. Cheng, M. Mccourt, Efficient rollout strategies for Bayesian optimization, in: *Proc. Conf. Uncertainty Artif. Intel.*, 2020, pp. 260–269.
- [93] A. Klein, S. Falkner, J.T. Springenberg, F. Hutter, Learning curve prediction with Bayesian neural networks, in: *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–11.
- [94] E. Yurtsever, J. Lambert, A. Carballo, K. Takeda, A survey of autonomous driving: Common practices and emerging technologies, *IEEE Access* 8 (2020) 58443–58469.
- [95] M. Sayed-Mouchaweh, E. Lughofer, Learning in non-stationary environments: methods and applications, Springer Science & Business Media, 2012.
- [96] Ó. Fontenla-Romero, B. Guijarro-Berdiñas, D. Martínez-Rego, B. Pérez-Sánchez, D. Peteiro-Barral, Online machine learning, in: *Efficiency and Scalability Methods for Computational Intellect*, IGI global, 2013, pp. 27–54.
- [97] F.M. Nyikosa, M.A. Osborne, S.J. Roberts, Bayesian optimization for dynamic problems, 2018, arXiv:1803.03432.
- [98] L. Wang, X. Zhang, H. Su, J. Zhu, A comprehensive survey of continual learning: Theory, method and application, 2024, arXiv:2302.00487.
- [99] A. Gepperth, B. Hammer, Incremental learning algorithms and applications, in: *Proc. Eur. Symp. Artif. Neural Netw.*, 2016, pp. 357–368.